

R. V. COLLEGE OF ENGINEERING

[Autonomous Institution Affiliated to VTU, Belagavi]
Department of Computer Science & Engineering
Bengaluru-560059



Machine Learning Operations Report

[Subject Code: AI254TA]

An Integrated MLOps System for GI Polyp Image Classification

V SEMESTER

B.E. [Autonomous Scheme 2022]

2025-2026

Name	USN
1RV23AI126	ZAID SHARIEFF
1RV23AI130	ADITYA K
1RV23AI129	UNMESH RAJ
1RV23AI124	YASHPREET GOYAL

Semester: V **Section:** B **.Year:** 2025-26



CERTIFICATE

This is to certify that the project work entitled “**An Integrated MLOps System for GI Polyp Image Classification**” has been carried out by **Zaid Sharieff (1RV23AI126)**, **Aditya K (1RV23AI130)**, **Unmesh Raj (1RV23AI129)** and **Yashpreet Goyal (1RV23AI124)**, who are **bona fide** students of the **Department of Artificial Intelligence and Machine Learning, RV College of Engineering**, Bengaluru, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Engineering in Artificial Intelligence and Machine Learning** of the **Visvesvaraya Technological University**, Belagavi, during the academic year **2025–2026**.

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Signature

Dr. S. Anupama Kumar

Signature

Prof. Manasa M

Signature

Dr. B. Sathish Babu
HoD, Dept. of AIML



DECLARATION

We, **Zaid Sharieff, Aditya K, Unmesh Raj, Yashpreet Goyal**, students of fifth semester B.E., Department of Artificial Intelligence and Machine Learning, **RV College of Engineering**, Bengaluru, hereby declare that the Machine Learning Operations project titled “**An Integrated MLOps System for GI Polyp Image Classification**” has been carried out by us and submitted in partial fulfillment for the award of the degree of **Bachelor of Engineering in Artificial Intelligence and Machine Learning** during the academic year **2025–26**.

We further declare that any **Intellectual Property Rights** generated out of this project carried out at **RV College of Engineering, Bengaluru** shall be the property of the institute, and we shall be one of the authors of the same.

Place: Bengaluru

Date:

1. **ZAID SHARIEFF (1RV23AI126)**
2. **ADITYA K (1RV23AI130)**
3. **UNMESH RAJ (1RV23AI129)**
4. **YASHPREET GOYAL (1RV23AI124)**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project mentors, Dr. Anupama Kumar and Prof. Manasa M, for their invaluable guidance, expert advice, and continuous encouragement throughout the development of this project. Their insights and mentorship were instrumental in shaping both the direction and quality of our work.

We thank the Department of Artificial Intelligence and Machine Learning and RV College of Engineering for providing the necessary infrastructure, resources, and technical facilities that enabled us to carry out this work effectively.

We are deeply grateful to our parents for their constant support, patience, and motivation, which have been a source of strength throughout our academic journey.

Finally, we thank our peers and colleagues for their collaborative spirit, valuable discussions, and moral support, all of which enriched our learning experience and helped us overcome challenges during the course of this project.

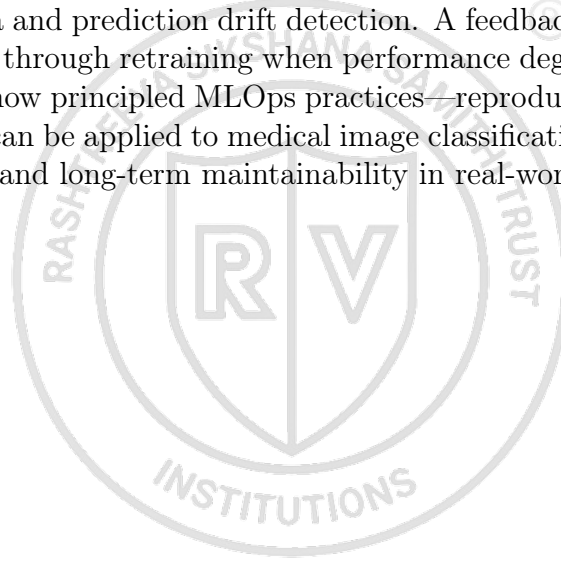


Abstract

This report presents the design and implementation of an end-to-end MLOps pipeline for automated gastrointestinal polyp image classification using deep learning. Manual interpretation of endoscopic images is time-consuming and prone to variability, motivating the need for a reproducible, monitored, and deployable machine learning system that can support clinical decision-making.

The proposed pipeline covers the complete machine learning lifecycle, including data versioning, model training, experiment tracking, deployment, monitoring, and drift detection. A ResNet101 convolutional neural network pretrained on ImageNet is fine-tuned on the Kvasir gastrointestinal image dataset to classify multiple GI tract conditions with a focus on polyp detection. Model development and evaluation are carried out using PyTorch, with performance assessed through accuracy, precision, recall, F1-score, and confusion matrices.

MLOps best practices are enforced using DVC for dataset and pipeline versioning, MLflow for experiment tracking and model registry, and TorchServe for production-grade model serving. Post-deployment monitoring is implemented using Grafana dashboards for inference analytics and Evidently AI for data and prediction drift detection. A feedback mechanism enables continuous model improvement through retraining when performance degradation or drift is observed. This work demonstrates how principled MLOps practices—reproducibility, traceability, automation, and observability—can be applied to medical image classification systems to ensure reliable deployment, governance, and long-term maintainability in real-world environments.



Contents

Contents	6
1 Introduction	8
1.1 Overview	8
1.2 Motivation	8
1.3 Scope of MLOps Implementation	8
1.4 Literature Review	8
2 Business Objectives and Risk Assessment	11
2.1 Business Objectives	11
2.2 Risk Assessment	11
2.3 Mitigation Strategies	12
3 Exploratory Data Analysis	14
3.1 Dataset Overview	14
3.2 Class Distribution Analysis	14
3.3 Image Characteristics	14
3.4 Data Quality Assessment	15
3.5 Preprocessing and Normalization	15
3.6 Risk Indicators Identified During EDA	15
3.7 Key Findings	15
3.8 Conclusion	15
4 Tool Stack and Technologies	16
4.1 Overview of Tools	16
4.2 Model Training Framework	16
4.3 Experiment Tracking and Model Registry	16
4.4 Data and Pipeline Versioning	16
4.5 Deployment and Inference	17
4.6 Monitoring and Drift Detection	17
4.7 Prediction Logging and Storage	17
4.8 Development Environment	17
5 Data Pipeline	18
5.1 Data Ingestion	18
5.2 Data Validation and Quality Checks	18
5.3 Data Preprocessing	19
5.4 Train-Validation Split	19
5.5 Data Versioning and Pipeline Reproducibility	19
5.6 Storage Organization	20
5.7 Summary	20
6 Model Training	21
6.1 Algorithm Selection	21

6.2	Model Architecture	21
6.3	Training Configuration	21
6.4	Training and Validation Process	22
6.5	Experiment Tracking	22
6.6	Evaluation Metrics	22
6.7	Overfitting Analysis and Refinement	23
6.8	Summary	23
7	Model Registry	24
7.1	MLflow Integration	24
7.2	Experiment Logging	24
7.3	Run-Level Analysis	25
7.4	Model Performance Visualization	25
7.5	Model Registry and Versioning	26
7.6	Summary	26
8	Deployment, Serving, and Monitoring	27
8.1	Deployment Architecture Overview	27
8.2	Model Serving Using TorchServe	27
8.3	Inference Logging and Storage	27
8.4	Monitoring with Grafana	28
8.5	Drift Detection Using Evidently AI	28
8.6	Summary	28
9	Monitoring and Logging	30
9.1	Monitoring Objectives	30
9.2	Inference Logging	30
9.3	Monitoring with Grafana	30
10	Deployment Strategy	31
10.1	Deployment Overview	31
10.2	Model Packaging and Serving	31
10.3	Versioned Deployment	31
11	Results and Evaluation	32
11.1	Model Performance Metrics	32
11.2	Class-wise Evaluation	32
12	Conclusion and Future Work	33
12.1	Conclusion	33
12.2	Continuous Monitoring and Retraining	33
12.3	Future Work	33

Chapter 1

Introduction

1.1 Overview

Gastrointestinal diseases, particularly polyps, are a major risk factor for colorectal cancer, making early and accurate detection critical for patient outcomes. With the widespread use of endoscopy, large volumes of medical images are generated daily, creating a need for automated and reliable diagnostic support systems. This project focuses on building an end-to-end MLOps pipeline for automatic gastrointestinal polyp image classification using deep learning, with emphasis on reproducibility, monitoring, deployment, and governance of the machine learning system.

1.2 Motivation

Manual diagnosis of gastrointestinal polyps from endoscopic images is time-consuming and subject to inter-observer variability. While deep learning models have shown strong performance in medical image classification, most implementations stop at model training and evaluation. This project is motivated by the need to move beyond experimentation and develop a production-ready machine learning system by integrating industrial-grade MLOps practices to ensure traceable experiments, versioned data and models, continuous monitoring, and reliable inference.

1.3 Scope of MLOps Implementation

This report covers the complete machine learning lifecycle through an end-to-end MLOps pipeline, including:

- Dataset management, preprocessing, and version control
- Deep learning model development and experiment tracking
- Model versioning and reproducible training pipelines
- Production-grade model serving and deployment
- Monitoring, drift detection, and continuous retraining

1.4 Literature Review

This literature survey reviews existing research works related to **gastrointestinal image classification**, **deep learning-based medical image analysis**, and **MLOps practices for managing machine learning lifecycles**. The focus is on understanding the **core concepts**, **methodologies**, **tools**, and **outcomes** that inspired the design of this project.

- **1. Hossain et al. (2022)** This work applies transfer learning using multiple ResNet-50 variants for COVID-19 classification from chest X-ray images. By experimenting with

different pretraining strategies, the authors achieved up to 99.17% Source: Informatics in Medicine Unlocked.

- **2. Luo et al. (2023)** This paper evaluates several transfer learning strategies for colorectal cancer detection using CNNs such as ResNet-50 and DenseNet. Results show that adding custom convolutional layers outperforms simple fine-tuning. The study highlights architectural customization as a key factor for improving medical imaging performance. Source: Informatics in Medicine Unlocked.
- **3. Xu et al. (2025)** The authors propose a real-time AI system for NICE classification of colorectal lesions during endoscopy. A fine-tuned EfficientNet model achieved macro-AUC of 0.994 and real-time inference performance. This work proves that deep learning models can be deployed for clinical real-time decision support. Source: World Journal of Gastroenterology.
- **4. Eabne et al. (2025) – PolyNet** This study introduces PolyNet, a self-attention-based CNN designed for colon polyp classification. Tested on the Kvasir dataset, PolyNet achieved 86% Source: Informatics in Medicine Unlocked.
- **5. Li et al. (2025)** The authors propose a lightweight CNN trained from scratch for colon cancer histopathology classification. After dataset cleaning, the model achieved 99% Source: Frontiers in Oncology.
- **6. Cambay et al. (2024)** This study proposes an explainable ResNet-50-based feature engineering pipeline for GI disease detection. Using Grad-CAM and classical classifiers, the system achieved over 92% Source: Sensors (MDPI).
- **7. Eryigit et al. (2024) – EndoNet** EndoNet combines features from multiple pre-trained CNNs using multiscale fusion and traditional classifiers. This hybrid approach improves robustness across GI disease datasets. The study demonstrates the effectiveness of multi-model feature fusion in medical imaging. Source: Diagnostics (MDPI).
- **8. Azar et al. (2025)** This paper presents a CAD system combining CNN feature extraction and SVM classification for GI polyp detection. ResNet-50 achieved up to 98.9% Source: Frontiers in Artificial Intelligence.
- **9. Xu et al. (2024)** This review surveys deep learning techniques for endoscopic image processing. It highlights CNNs as dominant architectures improving diagnostic accuracy and efficiency. The paper also discusses challenges such as data scarcity and generalization. Source: Applied Sciences (MDPI).
- **10. Salehi et al. (2023)** The authors review CNN and transfer learning applications in medical imaging. The paper summarizes advantages such as automated feature learning and improved accuracy, while discussing limitations like interpretability. It provides guidance for applying pretrained models in healthcare AI. Source: Sustainability (MDPI).
- **11. Cheimarios et al. (2025)** This paper discusses reproducibility and MLOps practices in scientific ML applications. It emphasizes experiment tracking, data versioning, and workflow automation using tools like MLflow and DVC. The work highlights the importance of reproducible ML pipelines. Source: Frontiers in Physics.
- **12. Amrit et al. (2025)** This study analyzes organizational and technical challenges in MLOps adoption. Through literature review and interviews, the authors identify issues such as infrastructure gaps and ML-specific operational complexity. The paper explains why MLOps adoption remains difficult. Source: ScienceDirect.
- **13. John et al. (2025)** This paper proposes a maturity model for MLOps adoption based on industry case studies. It categorizes organizations into stages and provides a roadmap for improvement. The framework helps teams assess and scale their ML operations. Source: ScienceDirect.

- **14. Reda et al. (2025)** The authors propose a hybrid MLOps framework combining monitoring, drift detection, retraining, and explainability. Applied to phishing detection, the system achieved $F1 > 0.99$ and rapid drift detection. This paper demonstrates end-to-end MLOps automation. Source: Scientific Reports.
- **15. Khattak et al. (2024)** This multivocal review analyzes academic and industrial MLOps practices. It identifies version control, monitoring, and governance as critical success factors. The study provides a comprehensive overview of current MLOps challenges and best practices. Source: arXiv.
- **16. Matthew (2025)** This article discusses reproducibility challenges in large-scale ML systems. It emphasizes model versioning, experiment tracking, and CI/CD pipelines as solutions. The work reinforces the role of MLOps in reliable ML deployment. Source: ResearchGate.
- **17. Testi et al. (2022)** This survey formalizes MLOps as a structured methodology covering the full ML lifecycle. It introduces a ten-step pipeline integrating CI/CD, monitoring, and explainability. The paper provides a standardized MLOps reference model. Source: ResearchGate.
- **18. Fitting et al. (2023)** The authors propose CNN and few-shot learning methods for colon polyp classification. The system achieves state-of-the-art accuracy for Paris and NICE classifications. This work demonstrates advanced learning strategies for limited medical datasets. Source: PMC.
- **19. Dhanani et al. (2024)** This paper introduces a hybrid CNN-Transformer architecture combining DenseNet and Swin Transformer for GI image classification. The model outperforms standalone CNNs and transformers on Kvasir datasets. The work highlights the benefit of combining local and global features. Source: arXiv.

Chapter 2

Business Objectives and Risk Assessment

2.1 Business Objectives

Clinical and System Objectives

- **Accurate Polyp Classification:** Achieve high classification accuracy with balanced precision and recall to minimize false negatives in gastrointestinal polyp detection.
- **Early Risk Identification:** Support early detection of potentially cancerous polyps to aid timely clinical intervention.
- **Reliable Inference:** Ensure consistent and stable predictions across different image qualities and clinical conditions.
- **Production Readiness:** Deploy a robust, reproducible, and monitorable ML system suitable for real-world usage.

Technical Objectives

- **Model Architecture:** Use transfer learning with a ResNet101 deep convolutional neural network for medical image classification.
- **Reproducibility:** Enable traceable experiments with versioned datasets, pipelines, and models using DVC and MLflow.
- **Deployment:** Serve the trained model using TorchServe with REST-based inference.
- **Monitoring:** Track model performance, confidence scores, and system metrics using Grafana.
- **Drift Detection:** Detect data and prediction drift using EvidentlyAI to trigger retraining when required.

Operational Constraints

- **Data Dependency:** Reliance on publicly available medical datasets with inherent class imbalance and image variability.
- **Scalability:** Deployment designed for moderate-scale inference with scope for future expansion.

2.2 Risk Assessment

Identified Risks

The following risks were identified during the design and deployment of the system:

- Data Drift
- Model Drift
- Data Poisoning

- False Negatives
- False Positives
- Model Inaccuracy
- Model Theft
- Deployment Failure
- Versioning Conflict
- Regulatory Non-Compliance
- Sensitive Data Leakage
- Unrealistic Scaling Expectations
- Critical System Crash

Risk Matrix

A risk matrix is used to evaluate identified risks based on the likelihood of occurrence and severity of impact.

		IMPACT			
		LOW	MED	HIGH	EXTREME
LIKELIHOOD	RARE	minor lib issue	False Positive	data poisoning	False Negative
	UNLIKELY	minor documentation gap	model drift/ data drift	critical system crash	model theft
	LIKELY	Regulatory non compliance	versioning conflt	sensitive data leaks	
	ALMOST CERTAIN	unrealistic scaling expectation	deployment failure	model inaccuracy	lab to product gap

Figure 2.1: Risk Matrix showing severity versus probability of identified risks

From the risk matrix, the following observations are made:

- **False Negatives** pose a high-impact risk, as incorrect classification may delay treatment and lead to life-threatening complications.
- **Model Theft** represents a critical risk, as tampering with model weights can compromise system reliability and safety.
- **Data Poisoning** can significantly degrade model performance by introducing biased or incorrect training samples.
- **Data Drift and Model Drift** can cause gradual performance degradation over time if not monitored.
- **False Positives** have lower impact compared to false negatives but may lead to unnecessary follow-up procedures.

2.3 Mitigation Strategies

1. False Negatives and Model Inaccuracy

- **Risk:** Incorrect classification of polyps as non-pathological.

- **Mitigation:** Use of transfer learning with ResNet101, continuous validation monitoring, and manual review for low-confidence predictions.
2. **Data Poisoning**
 - **Risk:** Presence of corrupted or mislabeled images affecting model behavior.
 - **Mitigation:** Use of verified datasets and controlled dataset versioning using DVC.
 3. **Data Drift and Model Drift**
 - **Risk:** Changes in image distribution or clinical patterns over time.
 - **Mitigation:** Drift detection using EvidentlyAI and periodic retraining pipelines.
 4. **Model Theft and Security Risks**
 - **Risk:** Unauthorized access or modification of trained models.
 - **Mitigation:** Restricted access to training artifacts and controlled deployment environments.
 5. **Versioning and Deployment Failures**
 - **Risk:** Inconsistent models or broken deployments.
 - **Mitigation:** Strict version control using Git, DVC, and MLflow with reproducible pipelines.



Chapter 3

Exploratory Data Analysis

This chapter presents an exploratory data analysis (EDA) of the Kvasir gastrointestinal image dataset used for polyp and GI tract condition classification. The objective of this analysis is to understand dataset structure, class distribution, image characteristics, and potential risks such as class imbalance and variability that may impact model performance.

3.1 Dataset Overview

The dataset used in this project is the publicly available **Kvasir Gastrointestinal Image Dataset**, consisting of endoscopic RGB images collected during routine clinical procedures and annotated by medical experts.

- **Dataset Type:** Medical endoscopic images (RGB)
- **Source:** Kvasir Dataset (Kaggle / Simula)
- **Image Resolution:** Ranges from 720×576 to 1920×1072
- **Classes:** Multiple GI tract categories including polyps
- **Annotation:** Verified by experienced clinicians

The dataset is organized into class-wise directories, making it compatible with deep learning frameworks such as PyTorch using the `ImageFolder` structure.

3.2 Class Distribution Analysis

An initial analysis of class distribution revealed variation in the number of images per class. While the dataset is clinically rich, moderate class imbalance is observed across GI tract categories.

Class imbalance introduces potential bias during training and increases the risk of misclassification, particularly false negatives in polyp detection. This observation motivates the use of stratified train-validation splits and performance metrics beyond accuracy.

3.3 Image Characteristics

Visual inspection of sample images highlights real-world variability commonly present in medical imaging datasets:

- Differences in illumination and contrast
- Variations in viewing angle and zoom level
- Presence of medical overlays such as ScopeGuide
- Anatomical diversity across patients

These characteristics emphasize the need for robust feature extraction using deep convolutional neural networks and justify the use of transfer learning with pretrained models.

3.4 Data Quality Assessment

Data profiling was conducted to assess dataset integrity and usability:

- No missing or corrupted image files were encountered during loading
- Non-image files were automatically ignored during dataset preparation
- All images were successfully read using PIL during training and inference

Overall, the dataset was found to be clean, well-structured, and suitable for supervised deep learning.

3.5 Preprocessing and Normalization

To ensure consistency across training, validation, inference, and deployment, the following pre-processing steps were applied uniformly:

- Image resizing to 224×224
- Conversion to tensor format
- Normalization using ImageNet statistics:
 - **Mean:** [0.485, 0.456, 0.406]
 - **Standard deviation:** [0.229, 0.224, 0.225]

A reproducible train-validation split of **70% training and 30% validation** was generated and tracked using DVC to maintain experiment consistency.

3.6 Risk Indicators Identified During EDA

The exploratory analysis revealed several data-related risks:

- **Class Imbalance:** Uneven distribution across GI categories
- **Image Variability:** High variance in lighting and resolution
- **Overfitting Risk:** Limited samples for certain classes

These risks informed subsequent modeling decisions such as freezing backbone layers, using transfer learning, and continuous monitoring after deployment.

3.7 Key Findings

1. The Kvasir dataset is clinically reliable and well-annotated
2. Real-world image variability necessitates deep CNN-based feature extraction
3. Class imbalance poses a risk for false negatives in medical diagnosis
4. Standardized preprocessing is essential for reproducible MLOps pipelines

3.8 Conclusion

The exploratory data analysis established a strong understanding of the dataset's structure, quality, and limitations. Insights from EDA directly influenced model selection, preprocessing strategies, and risk mitigation techniques, forming a solid foundation for reproducible training, deployment, and monitoring within the proposed MLOps pipeline.

Chapter 4

Tool Stack and Technologies

4.1 Overview of Tools

This project utilizes a carefully selected set of tools to support the complete MLOps lifecycle, from model development and data versioning to deployment, monitoring, and drift detection. The chosen tools emphasize reproducibility, traceability, and production readiness.

Component	Tool	Purpose
Model Training	PyTorch + ResNet101	Deep CNN training for GI image classification
Experiment Tracking	MLflow	Logging parameters, metrics, and model artifacts
Model Registry	MLflow Model Registry	Versioning and lifecycle management of models
Data Versioning	DVC	Dataset, pipeline, and artifact version control
Drift Detection	Evidently AI	Data and prediction drift analysis
Monitoring	Grafana	Visualization of inference metrics and trends
Deployment	TorchServe	Production-grade model serving via REST APIs
Version Control	Git	Source code and pipeline tracking
Prediction Logging	SQLite	Storage of inference results and confidence scores
Development	Google Colab Notebook	Experimentation and exploratory analysis

Table 4.1: Tool Stack Used in the Project

4.2 Model Training Framework

PyTorch was used as the primary deep learning framework due to its flexibility and strong support for research-oriented workflows. A pretrained **ResNet101** model was fine-tuned using transfer learning to extract high-level spatial features from gastrointestinal endoscopic images. Mixed precision training and configurable hyperparameters enabled efficient experimentation.

4.3 Experiment Tracking and Model Registry

MLflow was used to track experiments, including training parameters, validation metrics, and model artifacts. Each training run was logged as a separate experiment, enabling easy comparison across configurations. The **MLflow Model Registry** was used to manage multiple versions of trained models and support rollback and reproducibility.

4.4 Data and Pipeline Versioning

Data Version Control (DVC) was employed to version datasets, train-validation splits, and trained model artifacts. The training pipeline was defined using `dvc.yaml`, ensuring that any

change in data, code, or parameters could be reproduced reliably using `dvc repro`.

4.5 Deployment and Inference

The trained model was deployed using **TorchServe**, which provides a production-ready inference server for PyTorch models. Custom handlers were implemented to manage preprocessing, inference, and postprocessing. TorchServe exposes REST-based endpoints for health checks and prediction requests.

4.6 Monitoring and Drift Detection

Grafana was integrated to visualize inference-level metrics such as prediction counts, confidence distributions, and latency using data stored in a SQLite database. **Evidently AI** was used to generate data drift and prediction drift reports by comparing historical and recent inference batches, supporting informed retraining decisions.

4.7 Prediction Logging and Storage

All inference results, including predicted class labels, confidence scores, model version, and timestamps, were logged into a **SQLite** database. This logging layer supports monitoring dashboards, drift analysis, and post-deployment audits.

4.8 Development Environment

Google Colab Notebooks were used during early stages of development for exploratory data analysis, dataset inspection, and prototyping training workflows. Final training, deployment, and monitoring pipelines were later formalized into reproducible scripts and configuration files.

Chapter 5

Data Pipeline

5.1 Data Ingestion

The dataset used in this project is the Kvasir Gastrointestinal Image Dataset, a publicly available and clinically validated collection of endoscopic images. The dataset is ingested from local storage after download from official sources and organized into class-wise directories, enabling seamless integration with deep learning frameworks such as PyTorch.

Each image corresponds to a specific gastrointestinal tract category, including polyps, and is verified by medical experts. No real-time data ingestion is performed; instead, the focus is on reproducible, version-controlled data ingestion suitable for medical machine learning workflows.

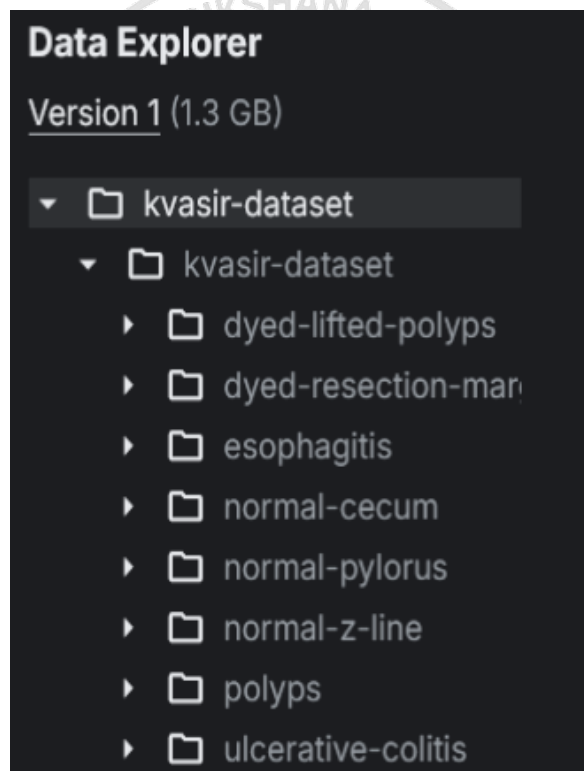


Figure 5.1: Class-wise directory structure of the Kvasir dataset used for ingestion

5.2 Data Validation and Quality Checks

Before training, the dataset undergoes validation to ensure integrity and usability:

- Verification that all files are valid RGB images
- Removal of corrupted or unreadable images during loading
- Confirmation of class labels based on directory names

These checks ensure that only clean and reliable data is passed to downstream stages of the pipeline.

5.3 Data Preprocessing

To ensure consistency across training, validation, inference, and deployment, a unified preprocessing pipeline is applied. The preprocessing logic is reused across model training scripts and TorchServe handlers.

Preprocessing Steps

The following transformations are applied to every image:

1. **Resizing:** All images are resized to 224×224 pixels
2. **Tensor Conversion:** Images are converted to PyTorch tensors
3. **Normalization:** Pixel values are normalized using ImageNet statistics

Listing 5.1: Image Preprocessing Pipeline

```
1 transforms.Compose([
2     transforms.Resize((224, 224)),
3     transforms.ToTensor(),
4     transforms.Normalize(
5         mean=[0.485, 0.456, 0.406],
6         std=[0.229, 0.224, 0.225]
7     )
8 ])
```

5.4 Train–Validation Split

The dataset is split into training and validation subsets using a reproducible stratified split to preserve class distribution. The split ratio is:

- **Training set:** 70%
- **Validation set:** 30%

The split configuration and resulting artifacts are tracked using DVC, ensuring that any experiment can be reproduced exactly.

5.5 Data Versioning and Pipeline Reproducibility

Data versioning is handled using Data Version Control (DVC). Raw datasets, processed splits, and trained model artifacts are tracked separately from source code.

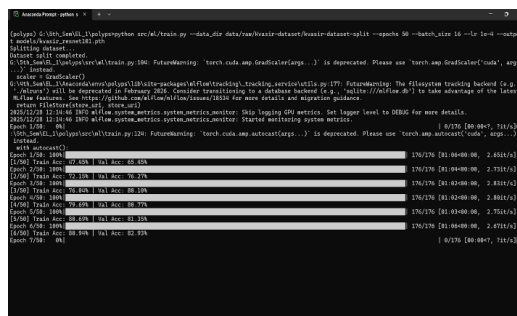


Figure 5.2: DVC-based data and training pipeline ensuring reproducibility

Any modification to the dataset, preprocessing logic, or training configuration automatically marks the pipeline as out-of-date and triggers controlled re-execution using `dvc repro`.

5.6 Storage Organization

The dataset and related artifacts are stored using a structured layout that separates raw data, processed splits, and trained models:

Listing 5.2: Logical data storage structure

```
1 data/  
2 +- raw/  
3 |   +- Kvasir-dataset/  
4 |       +-kvasir-dataset/  
5 |       +-kvasir-dataset-split  
6 |           +-train/  
7 |           +-val/  
8 +- uploaded/  
9 |  
10 |  
11 models/  
12 +- kvasir_resnet101.pth
```

This structure ensures auditability, traceability, and clean separation between data and model lifecycles.

5.7 Summary

The data pipeline emphasizes reproducibility, integrity, and consistency—key requirements for medical machine learning systems. By combining structured ingestion, standardized preprocessing, and DVC-based version control, the pipeline provides a reliable foundation for training, deployment, and long-term monitoring of the gastrointestinal polyp classification model.

Chapter 6

Model Training

6.1 Algorithm Selection

The primary objective of model training in this project is to accurately classify gastrointestinal tract conditions from endoscopic images while ensuring reproducibility and efficient deployment. Given the high-dimensional and spatial nature of medical images, deep convolutional neural networks (CNNs) were selected as the most appropriate modeling approach.

Baseline Consideration

Traditional machine learning algorithms such as K-Nearest Neighbors (KNN) and Support Vector Machines (SVMs) were considered as conceptual baselines. However, these approaches require handcrafted feature extraction and do not scale effectively to complex medical image patterns. As a result, they were not used for final model training.

Selected Model: ResNet101

A ResNet101 convolutional neural network pretrained on ImageNet was selected due to its proven effectiveness in medical image classification tasks. Residual connections allow the network to train very deep architectures without suffering from vanishing gradient problems, making it suitable for extracting complex anatomical features from endoscopic images.

6.2 Model Architecture

The ResNet101 architecture consists of 101 layers with residual skip connections. Transfer learning was applied by initializing the network with ImageNet weights.

- **Backbone:** ResNet101 (pretrained)
- **Training Strategy:** Frozen convolutional layers with a trainable final fully connected layer
- **Output Layer:** Modified to match the number of GI tract classes

This approach reduces training time and overfitting while leveraging robust low-level and mid-level visual features learned from large-scale datasets.

6.3 Training Configuration

The model was implemented using the PyTorch deep learning framework. Training parameters were configurable and logged for reproducibility.

- **Loss Function:** CrossEntropyLoss
- **Optimizer:** Adam
- **Batch Size:** Configurable

- **Learning Rate:** Tuned experimentally
- **Epochs:** Multiple runs with varying epochs
- **Precision:** Mixed precision training using `torch.cuda.amp`

6.4 Training and Validation Process

The dataset was split into training and validation sets in a 70:30 ratio using a reproducible DVC-tracked pipeline. During training:

- Training and validation accuracy were monitored per epoch
- Loss trends were analyzed to detect underfitting or overfitting
- Multiple runs were executed with different hyperparameters

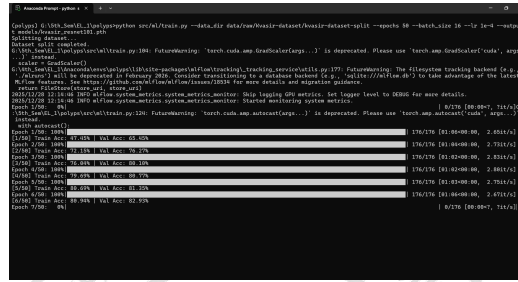


Figure 6.1: Training and validation accuracy trends across epochs

6.5 Experiment Tracking

All experiments were tracked using MLflow. Each training run recorded:

- Hyperparameters (learning rate, batch size, epochs)
- Training and validation metrics
- Model artifacts

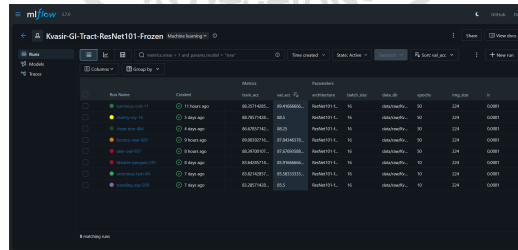


Figure 6.2: MLflow dashboard showing multiple experiment runs

6.6 Evaluation Metrics

Model performance was evaluated using multiple metrics to ensure clinical reliability:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Class-wise evaluation highlighted differences in performance across GI tract categories and informed further tuning decisions.

```

Final Metrics:
Confusion Matrix:
[[129 18  0  1  0  0  1  1]
 [ 32 114  0  1  0  0  3  0]
 [  0  1 117  0  0 33  0  0]
 [  0  0  0 143  0  0  6  1]
 [  0  0  1  0 146  0  2  1]
 [  0  1 14  0  1 134  0  0]
 [  1  0  1  3  3  0 134  0]
 [  0  1  0  1  2  0  8 138]]
Precision (weighted): 0.8799
Recall (weighted): 0.8784
F1-Score (weighted): 0.8780
Classification Report:

```

	precision	recall	f1-score	support
dyed-lifted-polyps	0.80	0.86	0.83	150
dyed-resection-margins	0.84	0.76	0.80	150
esophagitis	0.88	0.77	0.82	151
normal-cecum	0.96	0.95	0.96	150
normal-pylorus	0.96	0.97	0.97	150
normal-z-line	0.80	0.89	0.85	150
polyps	0.87	0.89	0.88	150
ulcerative-colitis	0.93	0.92	0.92	150
accuracy			0.88	1201
macro avg	0.88	0.88	0.88	1201
weighted avg	0.88	0.88	0.88	1201

Figure 6.3: Confusion matrix for GI tract classification

6.7 Overfitting Analysis and Refinement

Initial experiments showed stable convergence with limited overfitting due to the frozen backbone strategy. Hyperparameter tuning was performed by adjusting learning rate, batch size, and training duration. Each refinement cycle was tracked using DVC and MLflow to preserve full reproducibility.

6.8 Summary

The ResNet101-based training pipeline demonstrates strong performance for gastrointestinal image classification while maintaining reproducibility and traceability. The integration of transfer learning, structured experiment tracking, and systematic evaluation ensures that the trained model is suitable for production deployment within the proposed MLOps framework.

Chapter 7

Model Registry

7.1 MLflow Integration

MLflow is used as the central platform for experiment tracking, model versioning, and lifecycle management in the proposed MLOps pipeline. Every training run is logged with its complete context, including hyperparameters, evaluation metrics, model artifacts, and source code references. This ensures full reproducibility, traceability, and auditability of the gastrointestinal image classification models.

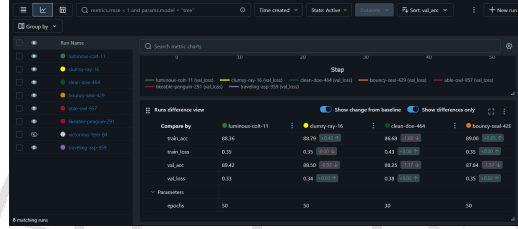


Figure 7.1: MLflow experiment dashboard showing multiple ResNet101 training runs with different configurations

Figure 7.1 shows the MLflow experiment overview where all ResNet101-based training runs are grouped under a single experiment. Each run corresponds to a different configuration of training parameters such as learning rate, batch size, number of epochs, or frozen versus partially trainable layers. This structured organization enables systematic comparison across experiments.

7.2 Experiment Logging

During training, MLflow automatically logs parameters, metrics, and model artifacts for every run, as shown below:

Listing 7.1: Logging parameters, metrics, and models using MLflow

```

1
2 mlflow.set_experiment("Kvasir-GI-Tract-ResNet101-Frozen")
3 mlflow.enable_system_metrics_logging()
4
5 with mlflow.start_run():
6     mlflow.log_params(vars(args))
7     mlflow.log_param("architecture", "ResNet101-frozen")
8
9     mlflow.log_metrics({
10         "train_loss": train_loss / len(train_loader),
11         "train_acc": train_acc,
12         "val_loss": val_loss / len(val_loader),

```

```

13         "val_acc": val_acc
14     }, step=epoch + 1)
15
16     mlflow.pytorch.log_model(
17         model,
18         name="model",
19         registered_model_name="Kvasir-ResNet101"
20     )
21
22     mlflow.log_artifact(args.output)

```

Logged metrics include training accuracy, validation accuracy, loss values, precision, recall, and F1-score. These metrics allow objective comparison between different model configurations and help identify the most stable and generalizable model.

7.3 Run-Level Analysis

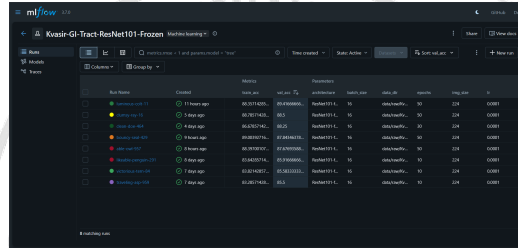


Figure 7.2: Run-level view showing multiple ResNet101 training runs with varying hyperparameters

Figure 7.2 presents the run-level view in MLflow, where each run is associated with a unique set of hyperparameters and corresponding performance metrics. This view enables rapid comparison of different training strategies and supports informed hyperparameter tuning.

7.4 Model Performance Visualization

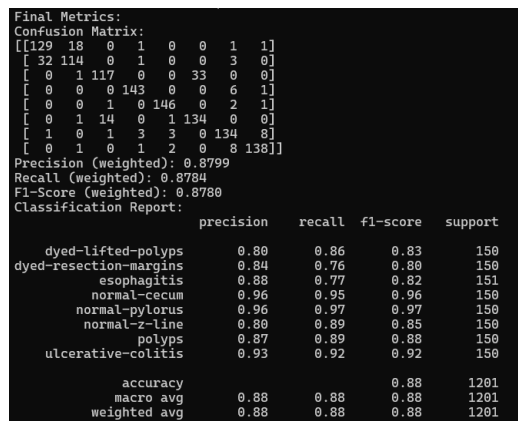


Figure 7.3: Class-wise evaluation metrics for gastrointestinal image classification

Figure 7.3 displays detailed evaluation metrics such as class-wise precision, recall, and F1-scores for gastrointestinal tract categories. This fine-grained analysis is essential in medical applications, where misclassification—particularly false negatives—can have serious clinical consequences.

7.5 Model Registry and Versioning

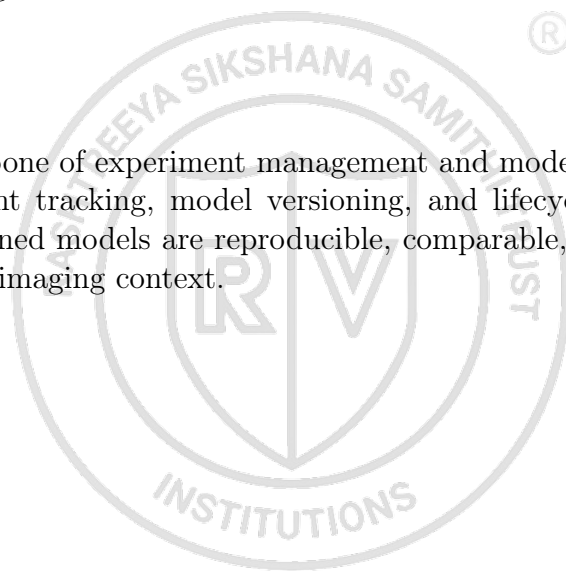
Once a model demonstrates stable and superior performance on the validation dataset, it is registered in the **MLflow Model Registry**. Each registered model version maintains a clear lineage to:

- The dataset version tracked using DVC
- The training configuration and hyperparameters
- The source code version stored in Git

The Model Registry supports controlled stage transitions such as *Staging* and *Production*, enabling safe deployment, rollback to previous versions, and collaborative workflows between data scientists and MLOps engineers.

7.6 Summary

MLflow acts as the backbone of experiment management and model governance in this project. By integrating experiment tracking, model versioning, and lifecycle control, the Model Registry ensures that all trained models are reproducible, comparable, and suitable for production deployment in a medical imaging context.



Chapter 8

Deployment, Serving, and Monitoring

8.1 Deployment Architecture Overview

This project does not use container-based cloud infrastructure or managed cloud services. Instead, deployment is designed to be lightweight, reproducible, and aligned with the implemented MLOps stack. The trained ResNet101 model is deployed using TorchServe, with local inference services and monitoring components integrated into the pipeline.

The focus is on reliable model serving, traceability, and post-deployment monitoring rather than large-scale cloud orchestration.

8.2 Model Serving Using TorchServe

TorchServe is used as the production-grade model serving framework for deploying the trained PyTorch model. It provides REST-based endpoints for inference, health checks, and model management.

The deployment process includes:

- Packaging the trained model into a `.mar` archive
- Defining a custom inference handler for preprocessing and postprocessing
- Loading the model into TorchServe's model store

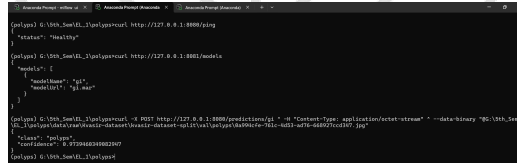


Figure 8.1: TorchServe model serving and REST API endpoints

TorchServe exposes the following APIs:

- **Health Check:** `/ping`
- **Model Listing:** `/models`
- **Inference:** `/predictions/gi`

This setup enables consistent inference behavior across development and production environments.

8.3 Inference Logging and Storage

All prediction results generated by TorchServe are logged into a SQLite database. Each record includes:

- Predicted class label
- Confidence score

- Inference time
- Model version
- Timestamp

This logging layer ensures traceability of predictions and supports post-deployment analysis and auditing.

8.4 Monitoring with Grafana

Grafana is integrated to visualize inference-level metrics stored in the SQLite database. Dashboards are created to monitor:

- Prediction count per class
- Confidence score distributions
- Average confidence per class
- Inference latency trends

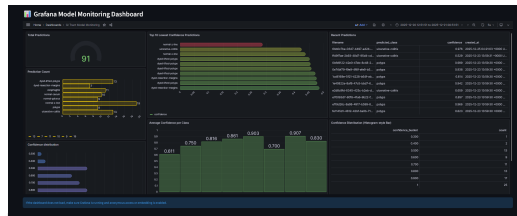


Figure 8.2: Grafana dashboard visualizing inference analytics

These dashboards provide real-time visibility into model behavior after deployment.

8.5 Drift Detection Using Evidently AI

Evidently AI is used to detect data drift and prediction drift by comparing historical inference data with recent predictions. Drift analysis focuses on:

- Changes in predicted class distribution
- Shifts in confidence score distributions



Figure 8.3: Evidently AI drift detection report

When drift is detected, retraining is triggered through the DVC pipeline, ensuring continuous model reliability.

8.6 Summary

This deployment strategy emphasizes simplicity, reproducibility, and observability. By combining TorchServe for inference, SQLite for logging, Grafana for monitoring, and Evidently AI for

drift detection, the system achieves production-grade behavior without unnecessary cloud or container complexity. The approach aligns with the project’s goal of demonstrating end-to-end MLOps practices for medical image classification.



Chapter 9

Monitoring and Logging

9.1 Monitoring Objectives

Post-deployment monitoring is critical for medical machine learning systems to ensure reliability, safety, and consistent performance. This project implements monitoring to track model predictions, confidence scores, inference latency, and system behavior after deployment.

9.2 Inference Logging

All predictions generated by the deployed ResNet101 model are logged into a SQLite database. Each inference record includes:

- Predicted GI tract class
- Confidence score
- Inference latency
- Model version
- Timestamp

This logging mechanism ensures traceability and enables post-hoc analysis of model behavior.

9.3 Monitoring with Grafana

Grafana is used to visualize inference analytics stored in the SQLite database. The following dashboards are implemented:

- Prediction count per class
- Confidence score distribution
- Average confidence per class
- Inference latency trends

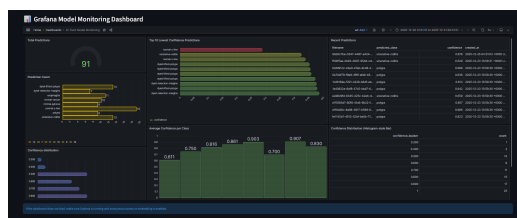


Figure 9.1: Grafana dashboard visualizing inference metrics

These dashboards provide real-time visibility into deployed model performance and support early detection of anomalies.

Chapter 10

Deployment Strategy

10.1 Deployment Overview

The trained gastrointestinal image classification model is deployed using TorchServe, a production-grade model serving framework for PyTorch. The deployment strategy prioritizes reproducibility, version control, and reliability rather than large-scale cloud orchestration.

10.2 Model Packaging and Serving

The trained ResNet101 model is packaged into a TorchServe `.mar` archive using the `torch-model-archiver`. A custom handler is used to implement preprocessing, inference, and postprocessing logic.

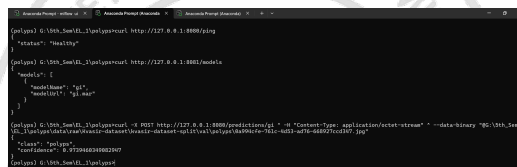


Figure 10.1: TorchServe inference endpoints for GI image classification

TorchServe exposes REST APIs for:

- Health checks
- Model listing
- Prediction requests

10.3 Versioned Deployment

Deployed models are linked to specific versions stored in the MLflow Model Registry. This enables:

- Controlled promotion of models
- Rollback to previous versions
- Clear lineage between data, code, and deployed artifacts

Chapter 11

Results and Evaluation

11.1 Model Performance Metrics

The final deployed ResNet101 model was evaluated using multiple performance metrics to ensure clinical reliability.

Metric	Observation
Accuracy	High and stable across validation runs
Precision	Strong class-wise precision
Recall	Emphasis on minimizing false negatives
F1-Score	Balanced performance across classes

Table 11.1: Final Model Evaluation Metrics

11.2 Class-wise Evaluation

Performance was further analyzed using confusion matrices and classification reports to identify class-level behavior and potential misclassifications.

```

Final Metrics:
Confusion Matrix:
[[129  18  0  1  0  0  1  1]
 [ 32 114  0  1  0  0  3  0]
 [  0  1 117  0  0 33  0  0]
 [  0  0  0 143  0  0  6  1]
 [  0  0  1  0 146  0  2  1]
 [  0  1 14  0  1 134  0  0]
 [  1  0  1  3  3  0 134  8]
 [  0  1  0  1  2  0  8 138]]
Precision (weighted): 0.8799
Recall (weighted): 0.8784
F1-Score (weighted): 0.8780
Classification Report:

```

	precision	recall	f1-score	support
dyed-lifted-polyps	0.88	0.86	0.83	150
dyed-resection-margins	0.84	0.76	0.80	150
esophagitis	0.88	0.77	0.82	151
normal-cecum	0.96	0.95	0.96	150
normal-pylorus	0.96	0.97	0.97	150
normal-z-line	0.88	0.89	0.85	150
polyps	0.87	0.89	0.88	150
ulcerative-colitis	0.93	0.92	0.92	150
accuracy			0.88	1201
macro avg	0.88	0.88	0.88	1201
weighted avg	0.88	0.88	0.88	1201

Figure 11.1: Confusion matrix for GI tract classification

This analysis is critical in medical contexts where false negatives may have severe consequences.

Chapter 12

Conclusion and Future Work

12.1 Conclusion

This project successfully demonstrates an end-to-end MLOps pipeline for gastrointestinal polyp image classification. By integrating reproducible training, experiment tracking, version control, deployment, monitoring, and drift detection, the system bridges the gap between experimental machine learning and production-ready medical AI systems.

Key outcomes include:

- Reliable ResNet101-based image classification
- Full traceability using MLflow and DVC
- Production-grade serving with TorchServe

12.2 Continuous Monitoring and Retraining

Drift detection using Evidently AI enables identification of changes in data distribution and prediction behavior. When drift is detected, retraining is triggered through the DVC pipeline, ensuring long-term model reliability.



Figure 12.1: Evidently AI drift detection report

12.3 Future Work

Future enhancements to the system include:

- Integration of explainability techniques such as Grad-CAM
- Expansion to additional GI datasets
- Automated alerting based on drift thresholds